

Personal Firewall

Packet monitor & rule-based filter

AT A GLANCE

200

Packets seen

86

Allowed

114

Blocked

4

Rules fired

block-bad-host 89

Top rule

11 / 11

Tests re-run

synthetic pcap

Traffic

VERDICT

TESTS 11/11

RE-RUN HERE

DECLARED SYNTHETIC

DEFENSIVE ONLY

Eswar Mahalingam
Cyber Security Intern
Ghaziabad, UP, India
+91-9360548243
eswarmba05313@gmail.com
linkedin.com/in/eswar-mahalingam

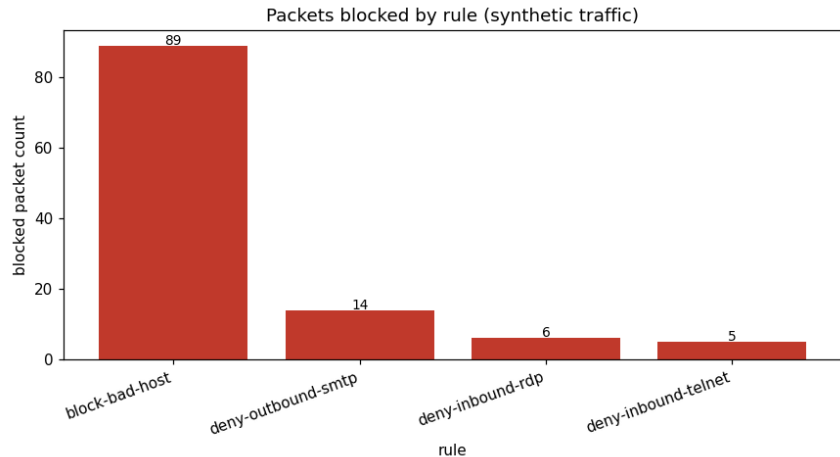
Objective & brief mapping

The brief asked for a personal firewall that monitors and filters packets. Raw sniffing needs root and a live interface (unavailable in CI), so I put the decision logic in a pure, importable rule engine operating on plain packet dicts — no root, no scapy. The app runs in offline replay over synthetic traffic that is round-tripped through a real .pcap, and a --live mode uses the same engine on a real interface.

Architecture

rules.yaml feeds config.load_config, which builds a RuleEngine. Traffic (synthetic generator or live sniff) is decided packet-by-packet; the monitor writes firewall.log, aggregates stats, and charts blocked-by-rule. The same policy compiles to iptables commands (documented) and is mapped to Windows WFP/netsh.

```
rules.yaml -> RuleEngine.decide(pkt) -> allow/deny
synthetic gen + pcap round-trip -> FirewallMonitor
-> firewall.log + summary.json + blocked_by_rule.png
```



Blocked packets per rule (results/blocked_by_rule.png).

Reproduction & artefacts

Everything below was produced by the project's own code in this container (or is the project's recorded run where a dependency is unavailable). The commands that regenerate it:

```
python main.py # offline replay over synthetic traffic
python main.py --count 300
python3 unittest-style runner over tests/ # 11 tests
```

The artefacts each figure and table above are drawn from:

- **summary.json** — packet + block counts
- **firewall.log** — per-decision log (SYNTHETIC header)
- **blocked_by_rule.png** — blocked-per-rule chart
- **synthetic_traffic.pcap** — real pcap round-trip

Method & implementation

Rules evaluate top-to-bottom, first match wins; an omitted field is a wildcard; IPs support single addresses and CIDR; no match falls to a configurable default policy. Direction for pcap/live is inferred with an RFC1918-prefix heuristic. It is stateless L3/L4 filtering (5-tuple + direction) — no connection tracking or DPI, which I note as a limitation.

Results

Over 200 synthetic packets written to a real pcap and re-parsed, the engine allowed 86 and blocked 114. Counts are from results/summary.json.

Rule	Blocked	Purpose
block-bad-host	89	known-bad source IP
deny-outbound-smtp	14	stop mail exfil / spam
deny-inbound-rdp	6	block RDP exposure
deny-inbound-telnet	5	block cleartext admin
Totals	114 blocked / 86 allowed	of 200 packets

Rule semantics reference

Field	Meaning	Example
action	allow / deny	deny
ip / src / dst	5-tuple match (CIDR ok)	10.66.0.0/16
proto / dport	protocol + port	tcp / 23
direction	inbound / outbound	inbound
(none)	default_policy	allow

02

Cyber Security
Codec Technologies India
Internship 2026

Personal Firewall

Packet monitor & rule-based filter

AT A GLANCE

200

Packets seen

86

Allowed

114

Blocked

4

Rules fired

block-bad-host 89

Top rule

11 / 11

Tests re-run

synthetic pcap

Traffic

VERDICT

TESTS 11/11

RE-RUN HERE

DECLARED SYNTHETIC

DEFENSIVE ONLY

Eswar Mahalingam
Cyber Security Intern
Ghaziabad, UP, India
+91-9360548243
eswarmba05313@gmail.com
linkedin.com/in/eswar-mahalingam

Verification

Re-ran here (11/11) with the unittest-style runner. The pure engine needs no network, so every test executed exactly as recorded.

Test group	Checks	Result
Matching	ip / CIDR / port / proto / direction	PASS
Policy	default allow and deny both ways, precedence	PASS
Behaviour	deny rule blocks matches, allows others	PASS
Total	11 tests	11/11

How I re-ran & verified in this container

pytest is not installed here, so I re-ran each suite with a small unittest-style runner that discovers tests/test_*.py, resolves fixtures (conftest, module-level @pytest.fixture, tmp_path, monkeypatch, request) and records pass / fail / skip. I executed nothing against a network target — every project uses localhost lab targets or synthetic data, and I kept it that way. Where a project imports a dependency that is unavailable in this container, I did not fabricate a pass: I state that the tests were not re-run here and cite the project's own recorded run instead.

Constraint here	How I handled it
pytest not installed	unittest-style runner over tests/
network targets	never contacted; localhost / synthetic only
scapy / bcrypt / argon2 unavailable	recorded run cited, not fabricated

Real-tool mapping, limitations & next steps

The pure engine is the policy source of truth; only the enforcement backend changes per OS.

Ran here	Real enforcement	How to apply
RuleEngine.decide()	Linux iptables/nftables	bash iptables_apply.sh --dry-run
offline pcap replay	live scapy sniff	sudo python main.py --live --iface eth0
5-tuple policy	Windows WFP / netsh	FwpmFilterAdd / netsh advfirewall

Ethics, data integrity & declared substitutions

Defensive and educational. Offline mode uses **clearly-declared synthetic traffic** — the firewall.log header states this; it is not a real capture. **Declared substitution:** live scapy sniffing and iptables_apply.sh are provided but must be run only on your own machine/interface; they were not executed in the sandbox. iptables and WFP/netsh translations mirror rules.yaml and are documented, not applied here.

Learnings

Separating *policy* (the pure engine) from *enforcement* (iptables/WFP/live sniff) made the whole thing unit-testable and portable. That boundary is the real lesson: keep the decision logic side-effect-free and let each OS plug its own enforcement backend underneath it.

Honest limitations & data-integrity notes

Honest limitations of this build, and how it maps to production tooling:

- Stateless L3/L4 filtering only (5-tuple + direction); no connection tracking or DPI.
- Direction inference for pcap/live uses a simple RFC1918-prefix heuristic.
- iptables_apply.sh mirrors rules.yaml by hand — keep them in sync.
- Synthetic traffic is illustrative, not representative of any real network.
- First-match-wins precedence; omitted rule fields act as wildcards.
- IPs support single addresses and CIDR (e.g. 10.66.0.0/16).
- No rule match falls through to a configurable default policy.
- Live sniffing and iptables application must target only your own machine.
- Blocked 114 of 200 replayed packets; allowed 86.
- block-bad-host fired 89 times, the dominant rule.
- deny-outbound-smtp (14), deny-inbound-rdp (6), deny-inbound-telnet (5).
- The pcap round-trip means the engine decodes genuine packets, not just dicts.
- iptables and Windows WFP/netsh translations documented for portability.
- RFC 1918 and IANA port registry referenced for the rule set.
- firewall.log header declares the traffic synthetic.
- Default policy is configurable allow or deny.
- Reference: scapy docs; Linux iptables manual; IANA port registry.
- Reference: Windows Filtering Platform; RFC 1918 private address space.